AGILE PROJECT DOCUMENTATION

# Multi-Agent AI News Assistant

Epic definition, features, user stories, acceptance criteria, task ownership, workflow, and delivery requirements

| | | | |
|---|---|---|---|
| **Epic ID** | **EPIC-01** | **Project type** | Generative AI / Multi-Agent System |
| **Primary language** | **Python** | **Orchestration** | LangGraph |
| **Team** | **3 developers** | **LLM access** | OpenRouter API (proposed) |

PROJECT OUTCOME

**A user provides a technology topic. The Research Agent gathers recent information from the web, the Summarization Agent produces five concise and source-backed points, and the Email Notification Agent creates a professional employee email and saves it as a .txt file. LangGraph coordinates the workflow.**

Prepared as a project baseline for Agile planning and implementation.

# Document Structure


- 1. Epic overview and problem statement alignment
- 2. Scope, objectives, assumptions, and exclusions
- 3. Feature breakdown and ownership
- 4. User stories, acceptance criteria, and implementation tasks
- 5. Agent input/output contracts
- 6. LangGraph orchestration and shared state
- 7. Non-functional requirements
- 8. Suggested project structure
- 9. Sprint plan, testing, and Definition of Done
- 10. Deliverables and Agile board hierarchy


# 1. Epic Overview

**Epic title:** Develop a Multi-Agent AI News Assistant

**Epic description:** As an employee, I want an AI-powered multi-agent assistant that automatically researches the latest developments in a user-provided technology topic, summarizes the research into five concise and informative points with source links, and generates a professional employee notification email saved as a text file, so that I can stay updated about emerging technologies without manually searching multiple sources.

## Problem statement alignment

The project addresses the effort involved in manually discovering, reviewing, summarizing, and communicating technology updates. It automates the workflow as a sequence of specialized agents: web research, AI summarization, and email notification. The final output is a plain-text email file that employees can access later.

## Business value

- Reduces time spent manually researching emerging technologies.
- Converts multiple research findings into five concise takeaways.
- Preserves source links so readers can verify the information.
- Automates preparation of a consistent employee notification.
- Demonstrates modular agent design and orchestration using LangGraph.

## Epic goal

Deliver a working end-to-end application that accepts a topic, gathers relevant recent web information, generates exactly five grounded key points when sufficient evidence exists, creates a professional email, and saves the email as a .txt file.

# 2. Scope and Requirements

## In scope

- Accept and validate a user-provided technology topic.
- Search the web using a search API or tool and collect multiple relevant sources.
- Capture titles, descriptions, source URLs, and publication dates where available.

- Summarize the findings into exactly five concise points when the evidence is sufficient.
- Associate source references with the relevant summary points.
- Generate a professional email subject and body containing all five points and links.
- Save the email as a .txt file in a designated output directory.
- Coordinate agents, state, and error handling using LangGraph.

### Out of scope for the initial version

- Automatically sending email through an email provider; the initial requirement is to generate and save a text file.
- User accounts, employee directory management, and role-based access control.
- A production web dashboard or scheduled recurring news delivery unless added as a later feature.
- Guaranteeing that every search result is factually correct; the system must preserve source references and avoid unsupported claims.

### Assumptions and dependencies

- A search API/tool and an LLM endpoint will be available, subject to valid credentials and usage limits.
- The team will use Python and LangGraph for the initial implementation.
- OpenRouter is a proposed LLM access provider; the team may select another compatible provider.
- All developers will agree on shared input/output data contracts before integration.
- Publication dates may not be available for every source and should be treated as optional.

## 3. Feature Breakdown and Ownership

| Feature ID | Feature | Description | Owner |
|---|---|---|---|
| **F-01** | **Web Research Agent** | Search the web for recent technology developments and return structured findings with source URLs. | Sreedev CS |
| **F-02** | **AI Summarization Agent** | Analyze research findings and produce five concise, grounded key points with source references. | Vishal Kumar Lal |
| **F-03** | **Email Notification Agent** | Generate a professional employee notification email and save it as a .txt file. | Vivek Sharma |
| **F-04** | **Multi-Agent Orchestration & Integration** | Coordinate the three agents using LangGraph, manage shared state, sequence execution, and handle errors. | All members |

### Feature F-01 — Web Research Agent

**Owner:** Sreedev CS

Develop an agent that accepts a validated technology topic, searches the web, collects relevant information from multiple sources, and passes structured research findings to the Summarization Agent.

**Expected output:** Topic, findings, source URLs, optional publication dates, status, and error details.

### Feature F-02 — AI Summarization Agent

**Owner:** Vishal Kumar Lal

Develop an agent that analyzes structured research findings and produces exactly five concise, professionally written points when sufficient valid research is available. Each point should retain appropriate source references.

**Expected output:** Topic, five key points, associated source links, status, and error details.

## Feature F-03 — Email Notification Agent

**Owner:** Vivek Sharma

Develop an agent that converts the five-point summary into a professional employee email with a subject, introduction, key points, source links, and closing. Save the result as a .txt file.

**Expected output:** Email subject, email body, output file path, status, and error details.

## Feature F-04 — Multi-Agent Orchestration & Integration

**Owner:** Shared responsibility — all three members

Integrate the independently developed agents using LangGraph. Define a shared state, connect nodes in the required order, validate outputs between stages, and handle failures without passing invalid data to downstream agents.

**Expected output:** A reliable end-to-end workflow that produces the notification text file when all stages succeed.

# 4. User Stories, Acceptance Criteria, and Tasks

Each user story describes a user or stakeholder need. Acceptance criteria define observable conditions that must be met before the story can be accepted. Tasks describe the implementation work required to deliver the story.

## US-01: Accept Technology Topic

**Owner:** Sreedev CS

**User story:** As an employee, I want to enter a technology topic so that the AI assistant can research the latest developments related to it.

### Acceptance criteria

- The system accepts a topic through user input.
- Empty or whitespace-only topics are rejected.
- A valid topic is passed to the Research Agent.
- Invalid input produces an appropriate error message.
- Relevant topics such as Generative AI, Agentic AI, Cloud Computing, and Cybersecurity can be entered.

### Implementation tasks

- T-01: Create the user input mechanism.
- T-02: Implement input validation.
- T-03: Pass the validated topic to the Research Agent.

## US-02: Search Latest Technology Developments

**Owner:** Sreedev CS

**User story:** As a user, I want the Research Agent to search the web so that I receive relevant and recent information about the selected technology.

### Acceptance criteria

- The agent performs web searches using a search API/tool.
- Information is collected from multiple relevant sources.
- Each finding includes a title, description, and source URL.
- Publication dates are captured wherever available.
- Results prioritize recent developments.
- Search failures are handled gracefully.
- Empty results are reported; the agent does not invent findings.
- Structured research is passed to the Summarizer Agent.

### Implementation tasks

- T-04: Select and configure a web search API/tool.
- T-05: Implement search functionality.
- T-06: Retrieve and extract relevant search results.
- T-07: Extract source URLs and available publication dates.
- T-08: Filter irrelevant or duplicate findings.
- T-09: Structure the research output.
- T-10: Implement search error handling.

## US-03: Summarize Research Findings

**Owner:** Vishal Kumar Lal

**User story:** As an employee, I want the AI assistant to summarize research findings into five concise points so that I can quickly understand the most important technology developments.

### Acceptance criteria

- The Summarizer Agent receives structured research findings.
- The agent analyzes the available information.
- It generates exactly five key points when sufficient valid research exists.
- Each point is concise, clear, and professionally written.
- The points represent important findings from the research.
- Repetition is avoided.
- Unsupported information is not introduced.
- Empty or insufficient research does not produce fabricated content.

### Implementation tasks

- T-11: Configure the LLM API and model.
- T-12: Create the Summarizer Agent.
- T-13: Design the summarization prompt.
- T-14: Process and analyze research output.
- T-15: Generate five key points.
- T-16: Validate summary structure and point count.
- T-17: Handle empty, incomplete, or invalid research input.

## US-04: Include Source References in Summary

**Owner:** Vishal Kumar Lal

**User story:** As an employee, I want source links included with the summarized information so that I can verify the original technology developments.

### Acceptance criteria

- Each summary point includes relevant source URLs or clearly identified references.
- Links correspond to original research findings.
- Duplicate source URLs are avoided.
- Missing or invalid URLs are not presented as verified references.
- Source information is preserved when passed to the Notifier Agent.

### Implementation tasks

- T-18: Map research sources to summary points.
- T-19: Validate source URLs.
- T-20: Remove duplicate references.
- T-21: Structure summary and references for downstream processing.
- T-22: Test source attribution and summary accuracy.

## US-05: Generate Employee Notification Email

**Owner:** Vivek Sharma

**User story:** As an employee, I want a professionally written technology update email so that I can stay informed about the latest industry developments.

### Acceptance criteria

- The Notifier Agent receives the validated summary.
- A professional subject line is generated.
- The email contains an appropriate introduction.
- The body includes all five key points.
- Relevant source links are included.
- A professional closing statement is present.
- The email is readable and properly formatted.

### Implementation tasks

- T-23: Create the Email Notification Agent.
- T-24: Design the email-generation prompt.
- T-25: Generate a suitable subject line.
- T-26: Generate the email body from the summary.
- T-27: Include all five points and references.
- T-28: Validate email structure and formatting.

## US-06: Save Email Notification as Text File

**Owner:** Vivek Sharma

**User story:** As an administrator, I want the generated employee notification to be saved as a text file so that it can be accessed and reviewed later.

### Acceptance criteria

- The email is saved in .txt format.
- The file contains the subject and complete email body.
- All five points and source links are preserved.
- The file is saved in the designated output directory.
- The filename is meaningful and identifiable.
- Successful file creation is confirmed.
- File-writing and directory errors are handled.

### Implementation tasks

- T-29: Create the output directory if it does not exist.
- T-30: Implement text-file generation.
- T-31: Save the email subject and body.
- T-32: Implement a meaningful filename convention.
- T-33: Handle file-system errors.
- T-34: Verify generated file contents.

## US-07: Coordinate Three AI Agents

**Owner:** Shared — all members

**User story:** As a developer, I want all three AI agents coordinated using LangGraph so that the complete research-to-notification workflow executes automatically.

### Acceptance criteria

- Three independent agents are implemented.
- LangGraph coordinates their execution.
- A shared state structure manages workflow data.
- Research output is passed to the Summarizer Agent.
- Summary output is passed to the Notifier Agent.
- Agents execute in the correct sequence.
- The workflow reaches file generation only after valid preceding outputs.
- Agent failures are propagated and handled.

### Implementation tasks

- T-35: Define the shared LangGraph state schema.
- T-36: Create the Research Agent node.
- T-37: Create the Summarizer Agent node.
- T-38: Create the Notifier Agent node.
- T-39: Define graph edges and execution sequence.
- T-40: Configure graph entry point and termination.
- T-41: Integrate individual agent modules.
- T-42: Test state transfer between agents.

### US-08: Handle Errors and Demonstrate Workflow

**Owner:** Shared — all members

**User story:** As a user, I want the application to handle execution errors and display appropriate results so that I can use the multi-agent assistant reliably.

#### Acceptance criteria

- Missing input is handled.
- Search API failures are handled.
- Empty research does not produce fabricated summaries.
- LLM/API failures are handled appropriately.
- Invalid agent outputs are detected.
- File-generation failures are reported.
- Successful execution produces the notification text file.
- The workflow can be demonstrated from user input to final output.

#### Implementation tasks

- T-43: Implement workflow-level exception handling.
- T-44: Add meaningful execution logs.
- T-45: Test individual agents independently.
- T-46: Perform end-to-end integration testing.
- T-47: Test failure scenarios and recovery behavior.
- T-48: Prepare sample test cases and demonstration.
- T-49: Prepare project documentation and setup instructions.

## 5. Agent Input and Output Contracts

The three agents are developed by different team members. They must agree on a shared data contract before integration. The examples below are illustrative Python dictionaries; the final implementation should use consistent field names and data types.

### Research Agent output (Agent 1 → Agent 2)

```
research_output = {
    "topic": "Generative AI",
    "findings": [
        {
            "title": "Technology development title",
            "description": "Research finding description",
            "source_url": "https://example.com/article",
            "published_date": "2026-11-10"
        }
    ],
    "status": "success",
    "error": None
}
```

## Summarizer Agent output (Agent 2 → Agent 3)

```
summary_output = {
    "topic": "Generative AI",
    "key_points": [
        {
            "point": 1,
            "summary": "First key development",
            "sources": ["https://example.com/article1"]
        }
    ],
    "status": "success",
    "error": None
}
```

The actual successful output must contain five key points. The shortened example above shows the structure only.

## Notifier Agent output

```
email_output = {
    "subject": "Technology Update: Generative AI",
    "body": "Dear Team,\n\nHere are the latest updates...",
    "file_path": "output/technology_update.txt",
    "status": "success",
    "error": None
}
```

## Integration agreement

- Use the same field names, data types, and status/error conventions across modules.
- Keep source URLs attached to the findings and summary points they support.
- Do not pass an empty or failed output to the next agent as if it were successful.
- Validate the structure at each agent boundary.
- Treat publication date as optional because search providers may not supply it.

# 6. LangGraph Orchestration and Shared State

LangGraph will coordinate the three specialist agents through a graph of nodes and transitions. Each agent remains an independently testable component, while a shared state object carries the topic, research findings, summary, email content, output path, status, and error information.

## Suggested shared state schema

```
from typing import TypedDict, Optional

class NewsAssistantState(TypedDict):
    topic: str
    research_results: list[dict]
    summary: list[dict]
    email_subject: Optional[str]
    email_body: Optional[str]
    output_file: Optional[str]
    status: str
    error: Optional[str]
```

## Suggested nodes and responsibilities

| Node | Responsibility |
|---|---|
| **Input / Validation** | Validate that the user supplied a non-empty topic. |
| **Research** | Call Agent 1 and store structured findings in shared state. |
| **Summarization** | Call Agent 2 and store five validated points and references. |
| **Notification** | Call Agent 3 to create the email content and save the text file. |
| **End / Result** | Return completion status, error details if any, and output file location. |

## Workflow sequence

**Start → Validate topic → Research Agent → Check research results → Summarizer Agent → Validate five-point summary → Email Notification Agent → Save .txt file → Return success.**

## Failure handling

- Invalid input: stop before the Research Agent and return a validation message.
- Search failure or no usable findings: stop before summarization and report the issue.
- Invalid or insufficient summary: do not generate an email that implies five verified points exist.
- LLM/API failure: record the failing stage and return a clear error.
- File-writing failure: report the file error and do not mark the workflow successful.

Retries should be limited to transient failures and should not create duplicate output files. Logs should identify the failing stage without exposing API keys or other secrets.

# 7. Non-Functional Requirements

| ID | Requirement | Description |
|---|---|---|
| **NFR-01** | **Reliability** | Handle API, agent, and file-system failures gracefully. |
| **NFR-02** | **Performance** | Avoid unnecessary repeated API calls and excessive processing. |
| **NFR-03** | **Maintainability** | Implement each agent as an independent Python module. |
| **NFR-04** | **Scalability** | Allow additional agents or workflow stages to be added later. |
| **NFR-05** | **Security** | Store API keys in environment variables; do not hardcode or commit secrets. |
| **NFR-06** | **Data integrity** | Preserve research source references through all workflow stages. |
| **NFR-07** | **Usability** | Show clear execution status and actionable error messages. |
| **NFR-08** | **Accuracy** | Ground summaries in collected findings and avoid unsupported claims. |

# 8. Suggested Project Folder Structure

Use a shared repository and keep the three agent modules separate. This makes individual development, independent testing, and integration easier.

```
multi_agent_news_assistant/
■■■ agents/
■   ■■■ research_agent.py
■   ■■■ summarizer_agent.py
■   ■■■ notifier_agent.py
■■■ orchestration/
■   ■■■ state.py
■   ■■■ graph.py
■■■ output/
■   ■■■ technology_update.txt
■■■ tests/
■   ■■■ test_research.py
■   ■■■ test_summarizer.py
■   ■■■ test_notifier.py
■   ■■■ test_workflow.py
■■■ .env
■■■ .env.example
■■■ .gitignore
■■■ requirements.txt
■■■ main.py
■■■ README.md
```

## Folder responsibilities

- **agents/:** Independent implementation of each specialist agent.
- **orchestration/:** Shared state definition and LangGraph workflow.
- **output/:** Generated employee notification files; generated outputs should be managed appropriately in version control.
- **tests/:** Unit tests for agents and end-to-end workflow tests.
- **.env:** Local secrets and credentials; must not be committed.
- **.env.example:** Names of required environment variables without real secret values.
- **requirements.txt:** Shared Python dependencies.
- **main.py:** Application entry point.

# 9. Sprint Plan

A two-sprint plan is proposed as an initial planning baseline. The team can adjust sprint duration and story estimates based on availability and project constraints.

## Sprint 1 — Individual Agent Development

**Suggested duration:** Week 1

- Sreedev: implement and test topic validation, web search, source extraction, and structured research output.
- Vishal: implement the summarization prompt, five-point output, source mapping, and validation.
- Vivek: implement email generation, formatting, text-file saving, and file error handling.
- All members: agree on the common data contracts and dependency versions.

**Sprint 1 exit criteria: each agent can be executed independently with sample input and returns the agreed output structure.**

## Sprint 2 — Integration and Testing

**Suggested duration:** Week 2

- Integrate the three agents using LangGraph.
- Implement shared state and conditional transitions for failure cases.
- Test state transfer and data validation between agents.
- Run end-to-end and negative test scenarios.
- Fix integration defects and prepare the final demonstration.
- Complete README and setup documentation.

**Sprint 2 exit criteria: a valid topic can flow through all agents and produce the final notification file; failure cases stop safely and provide useful error messages.**

# 10. Test Scenarios

| Test ID | Scenario | Expected result |
|---|---|---|
| **TC-01** | Enter a valid topic | Input is accepted and research starts. |
| **TC-02** | Enter an empty topic | Validation error; no search is performed. |
| **TC-03** | Search returns multiple results | Relevant findings and source URLs are collected. |
| **TC-04** | Search API fails | A research-stage error is returned; downstream agents do not run. |
| **TC-05** | No usable research findings | No fabricated summary is produced. |
| **TC-06** | Sufficient valid research | Exactly five grounded summary points are generated. |
| **TC-07** | Research contains duplicate URLs | Duplicate references are removed or deduplicated. |
| **TC-08** | LLM API fails | A clear agent error is returned. |
| **TC-09** | Valid five-point summary | Professional email subject and body are generated. |
| **TC-10** | Email generation succeeds | TXT file is created with subject and body. |
| **TC-11** | Output directory is missing | Directory is created if permitted; file is saved. |
| **TC-12** | File-writing fails | A file-stage error is reported; workflow is not marked successful. |
| **TC-13** | Complete valid workflow | Final notification file is generated and its path is returned. |

# 11. Definition of Done

## User Story Definition of Done

- All implementation tasks associated with the story are completed.
- The functionality satisfies the story's acceptance criteria.
- The code is committed to the shared repository and reviewed according to team practice.
- The agent or component has been independently tested.
- Required input/output formats are maintained.
- No hardcoded API credentials or committed secrets are present.
- Relevant error conditions are handled.
- The responsible team member has verified the implementation.

## Epic Definition of Done

- All three agents work independently.
- LangGraph successfully orchestrates the three agents.
- User input is accepted and validated.
- The Research Agent retrieves relevant information and source links.
- The Summarizer Agent generates exactly five grounded points when sufficient research is available.
- Source links are preserved in the summary and email.
- The Notifier Agent generates a professional email.
- The email is saved successfully as a .txt file in the designated output directory.

- Error-handling scenarios are tested.
- The end-to-end workflow demonstration is successful.
- README and project setup documentation are available.

# 12. Team Responsibility and Deliverables

| Team member | Primary responsibility | Task IDs | Deliverable |
|---|---|---|---|
| **Sreedev CS** | Web Research Agent | T-01 to T-10 | Research agent module and tests |
| **Vishal Kumar Lal** | AI Summarization Agent | T-11 to T-22 | Summarizer module and tests |
| **Vivek Sharma** | Email Notification Agent | T-23 to T-34 | Notifier module, TXT output, and tests |
| **All members** | Integration, testing, documentation | T-35 to T-49 | LangGraph workflow, test suite, README, demo |

# 13. Agile Board Structure

Recommended issue hierarchy for Jira or a similar Agile board:

- **Epic:** EPIC-01 — Multi-Agent AI News Assistant.
- **Features:** F-01 Web Research Agent; F-02 AI Summarization Agent; F-03 Email Notification Agent; F-04 Multi-Agent Orchestration & Integration.
- **User Stories:** US-01 through US-08, grouped under their corresponding features.
- **Sub-tasks:** T-01 through T-49, assigned to the responsible developer or shared team.

## Suggested workflow statuses

**To Do → In Progress → In Review → Testing → Done**

# 14. Team Coordination and Implementation Guidelines

- **Independent development:** Each developer builds and tests their agent independently before integration.
- **Common interface:** Agree on field names, types, and error conventions before coding the integration.
- **Shared environment:** Maintain a common requirements.txt and .env.example.
- **Version control:** Use separate feature branches and reviewed pull requests; protect the shared integration branch.
- **Integration responsibility:** All members participate in integration, with one agreed coordinator for the graph entry point and shared state.
- **Separation of concerns:** Research gathers information; summarization interprets the gathered findings; notification formats and saves the email.
- **Testing before merge:** Each agent must pass its independent tests before integration.
- **Credential safety:** Keep .env out of version control and never expose API keys in logs.

# 15. Final Success Criteria

The project is successful when a user enters a technology topic and the application executes the following sequence without manual intervention: the Research Agent gathers relevant recent developments from multiple sources; the Summarization Agent produces five concise, evidence-grounded points with source links; the Email Notification Agent creates a professional employee notification; and the system saves that notification as a .txt file. LangGraph manages the sequence, shared state, and failure paths.